Chapter 2: Network Models

A Student Study Guide — Layered Tasks, the OSI Model, TCP/IP, and Addressing

---

This guide walks through every concept in your Module 2 slides, in plain language, with explanations of *why* each idea matters — not just what it says.

1. Layered Tasks — The Big Idea

Before jumping into networking, the chapter uses a real-life analogy: sending a letter through the post office. This is meant to build intuition for why we split network communication into layers instead of doing everything at once.

Imagine you write a letter to a friend. You don't personally drive it to your friend's house. Instead:

- **Higher layer (you, the sender):** You write the letter, put it in an envelope, and drop it in a mailbox.
- **Middle layer (local transport):** The letter is carried from the mailbox to a post office.
- **Lower layer (long-distance carrier):** The post office hands the letter to a carrier (truck, plane) which transports it to the destination post office.

On the receiving end, the exact same steps happen in **reverse order**: the carrier delivers it to the destination post office, the post office delivers it to the mailbox, and finally your friend picks it up, opens the envelope, and reads it.

Why this matters for networking: Each 'layer' only worries about its own job. You don't need to know how the plane navigates, and the pilot doesn't need to read your letter. This separation of responsibility is exactly how computer networks are designed — each layer has a specific task, and layers don't interfere with each other's jobs. This is the foundation for the OSI model.

2. The OSI Model

ISO (International Organization for Standardization) is the organization — a global body that creates standards. **OSI** (Open Systems Interconnection) is the model that ISO created to standardize how network communication should be organized.

Remember the distinction the slide highlights:

> **ISO is the organization. OSI is the model.**

The OSI model breaks network communication into **7 layers**. Each layer has a clearly defined job, and each layer only talks to the layer directly above and below it.

2.1 The Seven Layers (top to bottom)

#	Layer	Main Job (in one line)
7	Application	Provides services directly to the user
6	Presentation	Translation, compression, and encryption of data
5	Session	Dialog control and synchronization between two systems
4	Transport	Reliable delivery of a message from process to process
3	Network	Delivery of individual packets from source host to destination host
2	Data Link	Moving frames from one hop (node) to the next

1	Physical	Moving individual bits over the physical transmission medium

Memory tip: A common mnemonic for remembering the order from Layer 7 down to Layer 1 is "*All People Seem To Need Data Processing*" — Application, Presentation, Session, Transport, Network, Data Link, Physical.

2.2 Peer-to-Peer Processes

Each layer on the sending device 'talks' conceptually to the **same layer** on the receiving device. This is called a peer-to-peer protocol. For example, the Transport layer on your computer follows the same rules as the Transport layer on the receiving computer, even though the actual bits physically travel down through all the layers, across the wire, and back up through all the layers on the other side. It *looks* like a direct conversation between matching layers, but physically the data goes down, across, and up.

2.3 Encapsulation

As data moves down through the layers on the sending side, each layer adds its own **header** (and the Data Link layer also adds a **trailer**). This wrapping process is called encapsulation.

- Layer 7 (Application) hands data down with header H7 added → becomes input for Layer 6.
- Layer 6 (Presentation) adds H6, Layer 5 (Session) adds H5, Layer 4 (Transport) adds H4.
- Layer 3 (Network) adds H3, forming a 'packet'.
- Layer 2 (Data Link) adds H2 *and* a trailer T2, forming a 'frame'.
- Layer 1 (Physical) converts everything into raw bits (0s and 1s) for transmission.

On the receiving side, this happens in reverse: each layer strips off its corresponding header/trailer as the data moves back up, until the original message reaches the receiving Application layer. This is called **decapsulation**.

3. Detailed Look at Each Layer

3.1 Physical Layer (Layer 1)

The physical layer is responsible for movements of individual bits from one hop (node) to the next.

This layer deals with the actual hardware: voltages, cables, radio signals, connectors. It doesn't care what the bits mean — only how to physically transmit a stream of 1s and 0s from one device to the next physical connection point.

3.2 Data Link Layer (Layer 2)

The data link layer is responsible for moving frames from one hop (node) to the next.

A 'frame' is data wrapped with a header (H2) and trailer (T2). This layer handles hop-to-hop delivery — meaning delivery between two directly connected devices (like your computer and the nearest router), not the full journey across the internet. It also handles error detection for that single hop.

Example — Hop-to-Hop Delivery: If a message travels from End System A → Intermediate System B → Intermediate System E → End System F, the data link layer manages each individual hop separately: A-to-B, B-to-E, and E-to-F. Each hop is its own separate delivery job.

3.3 Network Layer (Layer 3)

The network layer is responsible for the delivery of individual packets from the source host to the destination host.

Unlike the data link layer (which only handles one hop), the network layer is responsible for the **entire journey** — from the very first sending computer all the way to the final destination computer, even if the data passes through many intermediate routers along the way. This is called source-to-destination delivery. The Network layer is where **logical addressing (IP addresses)** comes in, since it needs a consistent way to identify the destination across the whole path.

3.4 Transport Layer (Layer 4)

The transport layer is responsible for the delivery of a message from one process to another.

This is a subtle but important distinction from the Network layer: the Network layer gets data from one **host (computer)** to another host, but the Transport layer makes sure it gets to the correct **process/application** running on that host (for example, making sure your web browser's data doesn't accidentally get delivered to your email app). This is also the layer responsible for breaking a message into smaller pieces called **segments**, and reassembling them correctly, including handling errors and lost data (reliability).

3.5 Session Layer (Layer 5)

The session layer is responsible for dialog control and synchronization.

"Dialog control" means managing whose turn it is to send data (like taking turns in a conversation). "Synchronization" means inserting checkpoints (shown as 'syn' in the diagram) into a long stream of data so that if a connection fails partway through a large transfer, it can resume from the last checkpoint instead of starting over completely.

3.6 Presentation Layer (Layer 6)

The presentation layer is responsible for translation, compression, and encryption.

- **Translation:** Converting data between different formats so two different systems (e.g. different operating systems using different character encodings) can understand each other.
- **Compression:** Reducing the size of data to make transmission faster.
- **Encryption:** Scrambling data for security so it can't be read if intercepted.

3.7 Application Layer (Layer 7)

The application layer is responsible for providing services to the user.

This is the layer closest to the actual human user — it's where programs like email clients, web browsers, and file transfer tools operate. Examples mentioned in the slides include X.500 (directory services), FTAM (file transfer/access/management), and X.400 (message handling).

4. The TCP/IP Protocol Suite

TCP/IP is the protocol suite that actually powers the real-world Internet (the OSI model is more of a theoretical/reference framework used for teaching and comparison). TCP/IP was originally described with 4 layers, but when compared side-by-side with OSI, it's usually shown as **5 layers**: Physical, Data Link, Network, Transport, and Application. Notice that TCP/IP **combines** OSI's Session, Presentation, and Application layers into a single Application layer — TCP/IP is less strict about separating those three responsibilities.

4.1 Key Protocols at Each Layer

Layer	Protocols	Purpose
Application	SMTP, FTP, HTTP, DNS, SNMP, TELNET	Email, file transfer, web browsing, name resolution, network management, r
Transport	SCTP, TCP, UDP	Process-to-process delivery (TCP = reliable/connection-based, UDP = fast/c
Network (Internet)	IP, ICMP, IGMP, ARP, RARP	IP handles addressing/routing; ICMP reports errors; ARP maps logical to ph
Data Link / Physical	Defined by underlying network hardware	Handled by the actual network technology in use (Ethernet, Wi-Fi, etc.)

5. Addressing in TCP/IP

An internet using TCP/IP uses **four different levels of addresses**, one corresponding to each major layer. Each address type exists because each layer has a different job to do, and needs a different way to identify 'who' it's talking to.

Address Type	Used At Layer	What It Identifies
Physical Address	Data Link / Physical	A specific hardware device on the local network (e.g. a MAC address)
Logical Address	Network	A host anywhere in the internet, independent of the physical network (e.g. a
Port Address	Transport	A specific process/application running on a host
Specific Address	Application	A human-friendly identifier such as an email address or a URL

The physical addresses will change from hop to hop, but the logical addresses usually remain the same.

Why physical addresses change but logical addresses don't: Every time data passes through a router on its way to the destination, the router replaces the source/destination physical (MAC) addresses with new ones relevant to the next hop in the path. But the logical (IP) address stays the same throughout the entire journey, because it identifies the ultimate source and destination — not just the next hop.

5.1 Worked Examples from the Slides

Example 2.1 — Physical Addresses:

A node with physical address 10 sends a frame to a node with physical address 87 over a shared bus LAN. Every device on the LAN receives the electrical signal, but each device checks the destination address in the frame header. If the destination address doesn't match its own address, it simply drops the packet. Only the device whose address matches (87) keeps and processes the frame.

Example 2.2 — Physical Address Format:

Most local-area networks use a 48-bit (6-byte) physical address, written as 12 hexadecimal digits separated by colons, e.g. **07:01:02:01:2C:4B**. This is the same format used for MAC addresses on real network cards.

Example 2.3 — IP (Logical) Addresses:

In an internet with two routers connecting three LANs, each computer has one pair of addresses (logical + physical) since it connects to only one network. Each router, however, has a separate pair of addresses for *every* network it connects to, because it acts as a bridge between multiple networks. As a packet crosses from LAN 1 through Router 1, across LAN 2, then through Router 2 to LAN 3, the physical addresses attached to the frame change at every hop, but the logical (IP) addresses of the true sender and receiver stay constant the whole way.

Example 2.4 — Port Addresses:

A sending computer may be running several processes at once (say, ports a, b, c), and the receiving computer may also be running multiple processes (ports j, k). If process 'a' on the sender needs to talk to

process 'j' on the receiver, the port addresses (a and j) travel unchanged from source all the way to destination — just like logical addresses — while only the physical addresses change at each hop.

Example 2.5 — Port Address Format:

A port address is simply a 16-bit number, so it can be represented as a single decimal number, e.g. **753**. (Real-world port numbers range from 0 to 65535 — this comes from 2^16 possible values.)

6. Quick Revision Summary

Use this section the night before your test — it condenses everything above into fast recall points.

6.1 Key Distinctions to Never Confuse

- **ISO vs OSI:** ISO is the organization; OSI is the model it created.
- **Data Link vs Network layer:** Data Link = hop-to-hop; Network = source-to-destination.
- **Network vs Transport layer:** Network = host-to-host; Transport = process-to-process.
- **Physical vs Logical address:** Physical changes every hop; Logical stays the same end-to-end.
- **OSI vs TCP/IP:** OSI has 7 layers (theoretical model); TCP/IP is usually shown with 5 layers (what actually runs the real Internet).

6.2 The Four TCP/IP Address Types (memorize this order)

Application → Specific address | Transport → Port address | Network → Logical address | Data Link/Physical → Physical address

6.3 Encapsulation Flow (Sender Side)

Message → + H7 (App) → + H6 (Presentation) → + H5 (Session) → + H4 (Transport, now a 'segment') → + H3 (Network, now a 'packet') → + H2 and T2 (Data Link, now a 'frame') → raw bits (Physical).

Compiled from Module 2: Network Models (Data Communication and Computer Networks). Use alongside your lecture slides for figure references (Fig. 2.1 – 2.21).